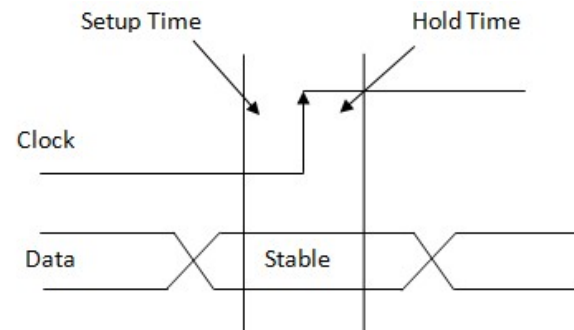


CDC

Setup and Hold time

- Setup time is **minimum time** the data signal must be **stable before** the **clock edge** arrives at the flip-flop.
- E.g. setup time = 1 ns, clock period = 10 ns, data must be stable at 9ns
- **Hold time** is the **minimum time** the data must remain **stable after** the **clock edge** has occurred.
- E.g. hold time = 0.5 ns, clock period = 10ns, data must not change for 0.5ns after clock edge.



Setup and Hold time

- Setup : Assume max delay data will take to reach from Launch flipflop to capture. Ensure data arrives before clock edge – setup even in worst case scenario.
- Hold : Assume min delay data will take to reach from Launch Flipflop to capture. Ensure don't change before hold time or too frequent.

Black

- No timing path between clock.
- This would appear if there is no logic or flop connection between signals clocked by clkA and those clocked by clkB.

```
always @(posedge clkA or posedge rst) begin
    if (rst)
        signalA <= 1'b0;
    else
        signalA <= trigger;
end
```

```
always@(posedge clkB or posedge rst)begin
    if (rst)
        signalB_sync <= 1'b0;
    else
        signalB_sync <= signalA;
end
```

Green

- Synchronous clocks with simple ratios. Vivado attempts to time them.
- If clkA and clkB are derived from the same PLL or have a known relationship, Vivado will try to fully time this path.
- Timed : **Vivado has enough information to perform full timing analysis** between the source and destination clocks.

Dark Blue

- All paths are ignored using constraints. Don't perform timings analysis on this path.
- Set_false_path / set_clock_groups –asynchronous.
- Best practice when clocks are asynchronous.

Light Blue

- Some paths are ignored, others not.
- You might only apply `false_path` to certain flip-flops or partial nets.

Red (Timed Unsafe)

- User provided clock constraints for async clocks and Vivado try to compute timings between them.

Why we ignore timings in async clock?

- The clocks can drift, causing metastability.
- The signal might change too close to clkB edge randomly.
- The "no violation" result is not meaningful.

Orange

- Same as Red, but you've added a `set_false_path` for few part of the CDC paths.

CDC

- When we are sampling signals from one clock domain to other its just wire without any logic elements leading to min. delay.
- for setup time, no logic will ensure data arrives well before next clock edge but this also have negative impact e.g. min. delay between two flipflop will be always less than hold time leading to hold violation.
- Hence, always implicitly specify two clocks are unrelated to avoid hold violation.

```
// clkA domain
always @(posedge clkA)
    signalA <= trigger;

// clkB domain
always @(posedge clkB)
    signalB <= signalA;
```

Clock Interaction Table

From Clock	To Clock	WNS Clock Edges	WNS(ns)	TNS(ns)	TNS Failing Endpoints	TNS Total Endpoints	WNS Path Requirement(ns)	Clock-Pair Classification	Inter-Clock Constraints
clkA	clkB	rise - rise	8.61	0.00	0	1	10.00	No Common Clock	Timed (unsafe)

Classification: "No Common Clock" means these two clocks are **asynchronous**, i.e., not derived from the same source and not phase-related.

Constraint: "Timed (unsafe)" → means the **tool is still analyzing** the timing path between them as if it's a valid synchronous path, which is **not safe**.

Destination Clocks

clkB

A solid red block between clkA and clkB means the tool is **timing** this cross-domain path and considers it **unsafe**.

Note: Zoom in to see the clock names if they are not displayed

■ No Path
■ User Ignored Paths

■ Timed
■ Timed (unsafe)

■ Partial False Path
■ Partial False Path (unsafe)

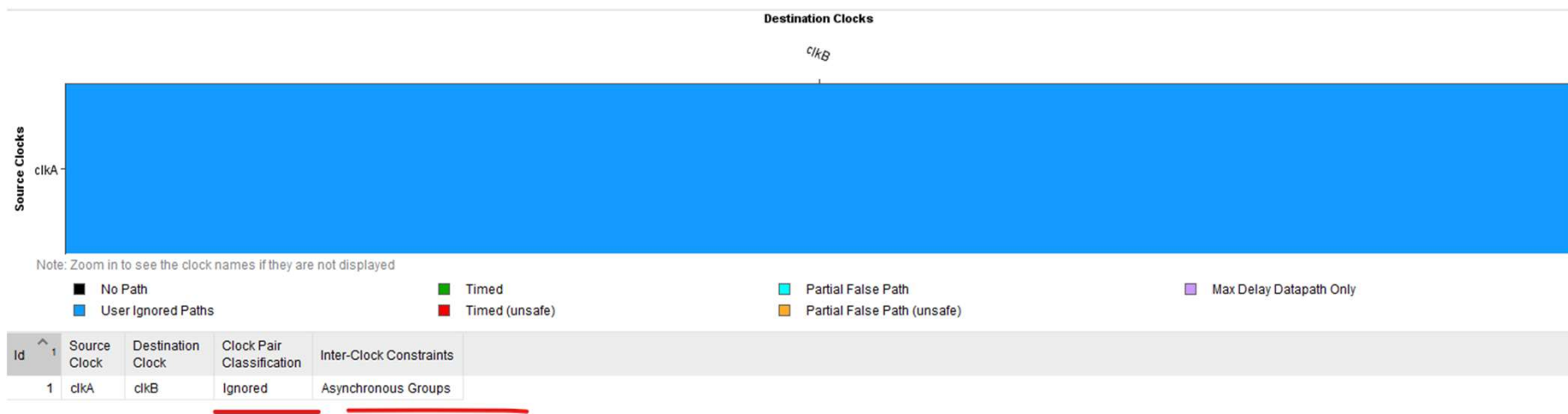
■ Max Delay Datapath Only

Id	Source Clock	Destination Clock	Edges (WNS)	WNS (ns)	TNS (ns)	Failing Endpoints (TNS)	Total Endpoints (TNS)	Path Req (WNS)	Edges (WHS)	WHS (ns)	THS (ns)	Failing Endpoints (THS)	Total Endpoints (THS)	Path Req (WHS)	Clock Pair Classification	Inter-Clock Constraints
1	clkA	clkB	rise - rise	8.609	0.000	0	1	10.000	rise - rise	-0.093	-0.093	1	1	0.000	No Common Clock	Timed (unsafe)

Asynchronous clock

- These two clocks (clkA and clkB) are asynchronous — they have no predictable timing relationship.
- Do not analyze timing between these two domains.
- The tool assumes clkA and clkB might be synchronous. It tries to compute setup and hold timing between flip-flops in different clock domains.
- This can lead to **false setup or hold violations**, especially for **short paths**.
- Whenever crossing data between these clocks using cdc technique, specify clocks are async to convey **vivado timing analyzer to skip** setup/hold analysis for these paths.

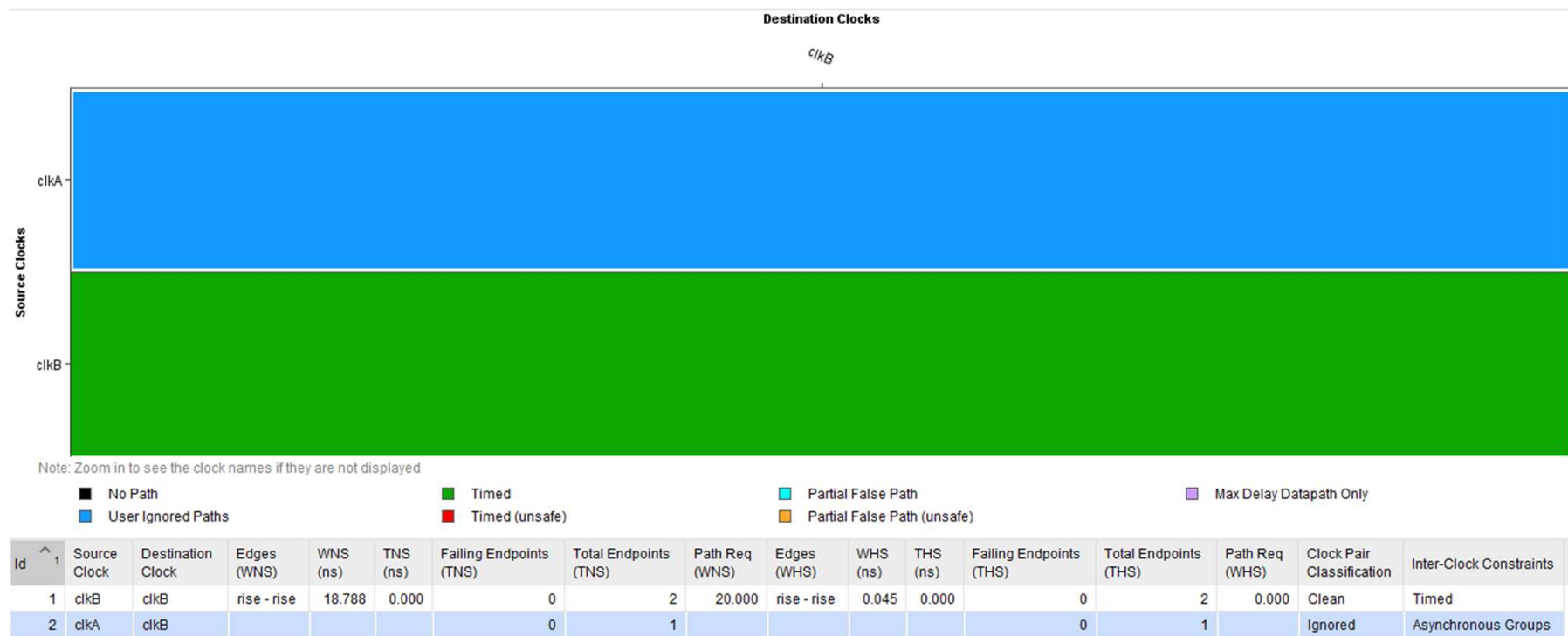
```
set_clock_groups -asynchronous -group {clkA} -group {clkB}
```



Color	Meaning	When It Happens
Red	Timed (unsafe) → Tool is analyzing timing	When clocks are asynchronous but not constrained
Green	Timed (safe) → Valid timing path	Between properly related clocks
Blue	User Ignored Paths → Not analyzed	When <code>set_clock_groups -asynchronous</code> is used
Black	No path exists	No data flows between the two clocks

Clock interaction

- Shows clocks interaction with each other across the design, using color-coded classifications. E.g. If your design uses clkA for logic and clkB for memory, the report shows if they interact.
- Helps identify where clock domain crossings may occur and if they are properly constrained or synchronized. E.g. you send data from clkA to clkB.
- Confirms whether clocks are defined as synchronous, asynchronous, or ignored based on applied constraints. E.g. explicitly specify clkA and clkB are unrelated.
- Ensures only valid timing paths are analyzed, avoiding false violations between asynchronous clock domains. E.g. avoid false setup/hold violations between clkA and clkB.
- Quickly detects unexpected clock interactions or missing constraints that may lead to timing or CDC issues. E.g. feed signal from one clock to other without sync
- Serves as a first step before detailed CDC analysis, guiding where to focus synchronizer checks. E.g. Before running a report_cdc, you check this report to see all crossing pairs



Id	Source Clock	Destination Clock	Edges (WNS)	WNS (ns)	TNS (ns)	Failing Endpoints (TNS)	Total Endpoints (TNS)	Path Req (WNS)	Edges (WHS)	WHS (ns)	THS (ns)	Failing Endpoints (THS)	Total Endpoints (THS)	Path Req (WHS)	Clock Pair Classification	Inter-Clock Constraints
1	clkB	clkB	rise - rise	18.788	0.000	0	2	20.000	rise - rise	0.045	0.000	0	2	0.000	Clean	Timed
2	clkA	clkB				0	1					0	1		Ignored	Asynchronous Groups

Typical use

- It shows **every pair of clocks** in your design and whether signals are passing between them.
- Data Cross clock domain -> missing constraints
- Data cross clock domain -> Async -> Ignore -> Proper CDC Sync

Report_cdc

- Clock Domain Crossings (CDC) report performs a structural analysis of the clock domain crossings in your design.
- Structural analysis only take into account how signals are feed to flipflop without considering any timings.
- Verify how signal crosses domains.
- You can use this information to identify potentially unsafe CDCs, which will lead to metastability or data coherency issues.
- Before generating the CDC report, you must ensure that the design has been properly constrained and there are no missing clock definitions.

What report cdc will analyze

- On all paths between asynchronous clocks.
- If clkA and clkB are marked as asynchronous (using `set_clock_groups`), and signal crosses from clkA domain to clkB domain then `report_cdc` will analyze that path to ensure proper synchronizer are used.

What report cdc will analyze

- If two clocks are synchronous and there are no timing exceptions (like `false_path`), then setup and hold checks by the timing engine are sufficient to ensure safe data transfer.
- **No need for special CDC logic** like synchronizers — as long as proper timing constraints are in place and timing is met.
- **Paths between synchronous clocks** — **only** if exceptions such as `set_false_path` or `set_clock_groups` is used.

Safe & Unsafe terminology

- an asynchronous crossing is safe when proper synchronization circuitry is used to prevent metastability. For example, a safe single-bit CDC can be implemented by a synchronizer, which is a chain of registers with same clock and control signals.
- A safe multi-bit CDC can be implemented with a MUX Hold circuitry or a Clock Enabled Controlled circuitry.
- an unsafe CDC is when the CDC analysis engine does not recognize a known safe synchronization circuit on an asynchronous CDC path.

CDC

Report_cdc

- `report_cdc` : print summary of clock pair
- `report_cdc -details` : print all details that you see in GUI mode
- `report_cdc -name cdc_report` : detailed GUI report
- GUI report generation

report_cdc

CDC Report

Severity	Source Clock	Destination Clock	CDC Type	Exceptions	Endpoints	Safe	Unsafe	Unknown	No ASYNC_REG
Critical	clkA	clkB	No Common Primary Clock	Asynch Clock Groups	1	0	0	1	0

Severity	Critical	The tool found a potentially unsafe or unverified CDC crossing
Source Clock	clkA	Data is launched from clkA domain
Destination Clock	clkB	Data is captured in clkB domain
CDC Type	No Common Primary Clock	Clocks are asynchronous, as expected
Exceptions	Asynch Clock Groups	You used set_clock_groups -asynchronous , so STA skips this path
Endpoints	1	One CDC signal is found
Safe	0	Tool could not confirm this CDC is handled safely
Unsafe	0	Tool also didn't find outright unsafe conditions
Unknown	1	It can't verify whether this path is safe or not
No ASYNC_REG	0	No flip-flop with ASYNC_REG property missing —

- **Severity:** Reports the worst severity of all CDC paths from/to the listed clocks. Values are Info, Warning, or Critical.
- **Source Clock:** Shows the name of the CDC source clock.
- **Destination Clock:** Shows the name of the CDC destination clock.

- **Critical :** No synchronizer are found between crossing.
- **Warning :** Synchronizer structure could be improved.
- **Info :** Safe

- **CDC Type:** Reflects the relationship between two clocks and their dominant timing exception, if any. Possible types are:
 - **Safely Timed:** All CDC paths are safely timed because the clocks are synchronous and accurate timing is not prevented by a timing exception.
 - **User Ignored:** All CDC paths are covered by `set_false_path` or `set_clock_groups`.
 - **No Common Primary Clock:** The CDC clocks are asynchronous and at least 1 CDC path is normally timed between two clocks that do not have a common primary clock.
 - **No Common Period:** The CDC clocks are asynchronous and at least 1 CDC path is normally timed between two clocks that do not have a common period. For the definition of clocks with no common period, refer to [Timing Analysis Key Concepts](#).
 - **No Common Phase:** The CDC clocks are asynchronous because there is no known phase relationship between the two clocks.

- Safely timed : Synchronous without exceptions.
- User ignored : Async with exceptions.
- No common primary clock : Async clocks

- **Exceptions:** The timing exceptions applied to the CDC (if any) are:
 - **None:** No clock group/false path/max delay datapath only timing exceptions exist on the CDC paths. Other timing exceptions such as multicycle paths, min delay, and max delay are not reported by `report_cdc`.
 - **Asynch Clock Groups:** The `set_clock_groups -asynchronous` exception was applied to the CDC clocks.
 - **Exclusive Clock Groups:** The `set_clock_groups -exclusive` exception was applied to the CDC clocks.
 - **False Path:** The `set_false_path` exception was applied to from/to the CDC clocks or to all CDC paths.
 - **Max Delay Datapath Only:** The `set_max_delay -datapath_only` exception was applied to all CDC paths. Note that Max Delay Datapath Only is reported when at least one CDC path is only covered by `set_max_delay -datapath_only`, while all other CDC paths are ignored due to `set_false_path` constraints.
 - **Partial Exceptions:** A mix of `set_false_path` and `set_max_delay -datapath_only` constraints are applied to some of the CDC paths, and at least one CDC path is normally timed.

Exceptions

- None – no exceptions – full timing on CDC analysis
- Async clock groups – Async exception is used for clock pairs
- Exclusive Clock group – Only one clock will be active at a time
- False – set_false_path is for clock paths
- Max Delay Datapath Only – Analyze only combinational delays between paths ignore setup & hold check
- Partial – paths with & without exceptions

- **Safe:** The number of safe CDC path endpoints. Safe endpoints are endpoints on CDC paths identified as:
 - Asynchronous clocks with known safe CDC structures
 - Synchronous clocks with exceptions and known safe CDC structures
 - Synchronous clocks without exceptions that are safely timed regardless of the CDC structure
 - CDC synchronized with `HARD_SYNC` macro
- **Unsafe:** The number of CDC path endpoints that are recognized as having an unsafe structure. The unsafe endpoints are CDC-10, CDC-11, CDC-12 and CDC-13.
 - Combinatorial logic topology
 - Fanout topology
 - Multi-clock fan-in topology
 - Non-FD primitive topology
- **Unknown:** The number of unknown CDC path endpoints. No CDC structure can be matched on these endpoints or an unknown CDC circuitry has been detected (CDC-1, CDC-4 and CDC-7).
- **No `ASYNC_REG`:** The number of identified synchronizers that are missing the `ASYNC_REG` property on at least one of the two first FD cells of the chain.

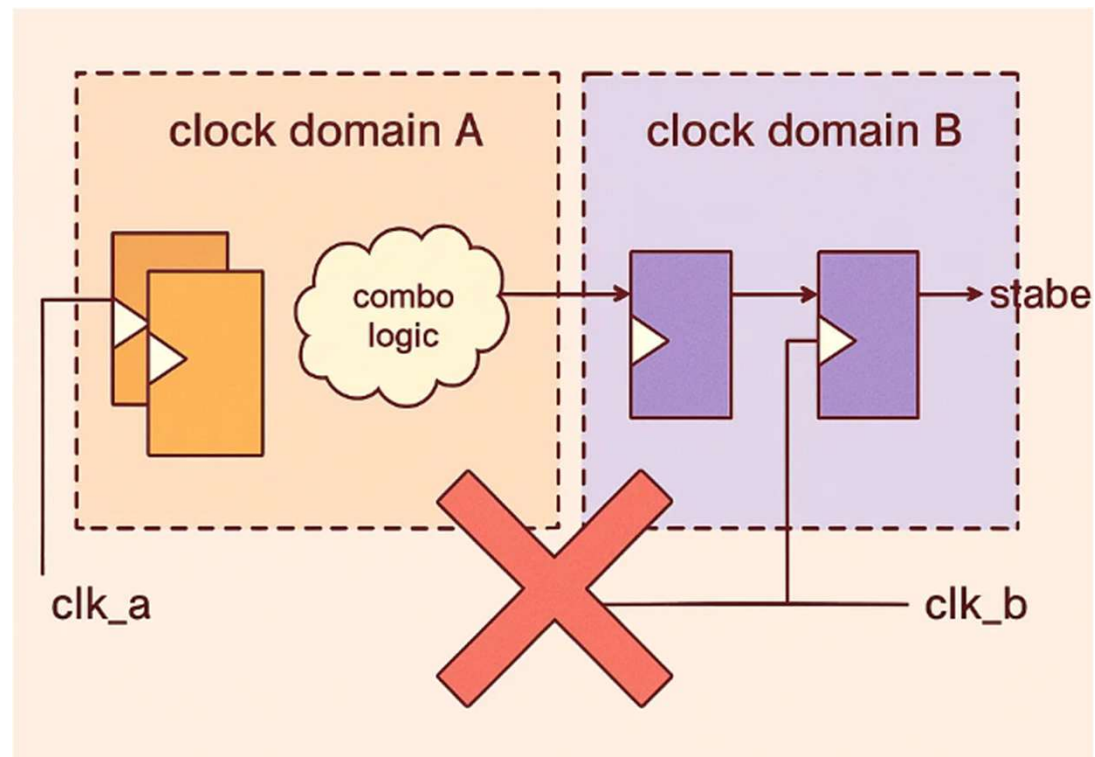
CDC Report

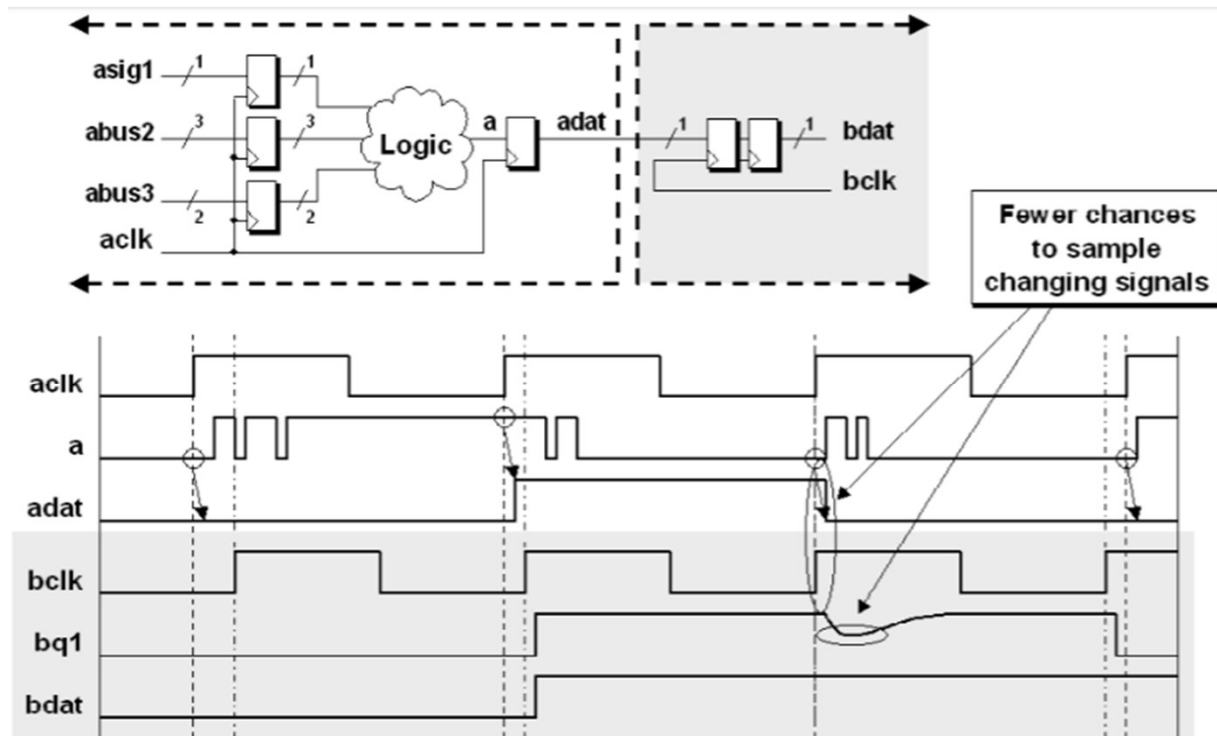
Severity	Source Clock	Destination Clock	CDC Type	Exceptions	Endpoints	Safe	Unsafe	Unknown	No ASYNC_REG
Critical	clkA	clkB	No Common Primary Clock	Asynch Clock Groups	1	0	0	1	0

- Report_cdc – show status of CDC structure used for crossing signal and safe to avoid metastability

Combinational logic at end

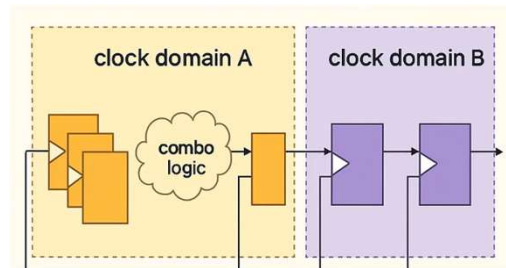
- If we send a **combinational signal** (not registered) from one clock domain to another, it might not be stable at the clock domain boundary.
- As the logic "settles" (finishes switching), it can briefly create **glitches or fast-changing pulses**.
- The signal might change multiple times very quickly.
- The receiving clock could **sample the signal while it's still changing**.
- This increases the risk of **metastability** (unpredictable logic behavior).





Reference : Clifford C. "Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog"

Precautions



- You should not directly connect a signal from clock domain A to a synchronizer in clock domain B, because the signal might have small glitches due to combinational logic.
- To avoid problems, you should **register the signal in clock domain A first**, and then connect it to the synchronizer in clock domain B.
- This reduces how often the signal changes, which lowers the chance of the receiving side sampling it while it's changing.
- As a result, it improves reliability by **increasing the time between possible failures** due to metastability.

CDC

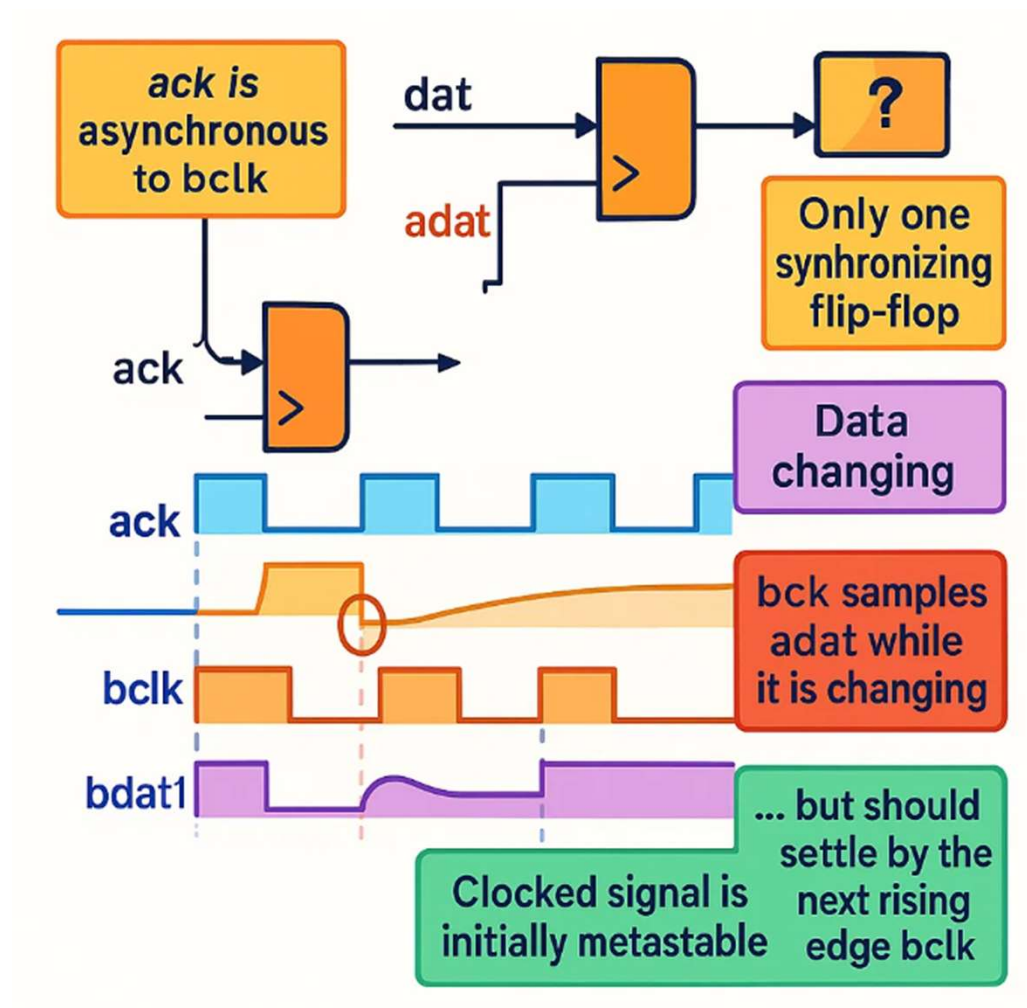
- **CDC (Clock Domain Crossing)** ensures data is **safely transfer between different clock domains** within the FPGA.
- In real-world FPGA systems, not all parts of a design operate at the same clock frequency. e.g. If a UART interface runs at 9600 baud (very slow) and your main system clock is 100 MHz, you need CDC logic to safely move UART data into your fast logic domain.
- Transferring data directly between different clock domains **without synchronization** can lead to Metastability, Data Corruption, Glitches etc

Metastability

- We assume signals are always either **logic 0** (low voltage) or **logic 1** (high voltage).
- But at the physical level, every signal is made of **voltages**, and sometimes those voltages don't clearly fall into 0 or 1.
- Input voltage must be either firmly in the logic 0 range (e.g., $< 0.4\text{ V}$), or Firmly in the logic 1 range (e.g., $> 0.8\text{ V}$) before Setup time.
- if the input voltage is between 0.4 V to 0.8 V in a 1.2 V logic system), flip-flop enter a **metastable state** — where the output voltage is unstable, not a valid 0 or 1, and it may take **extra time** (nanoseconds) to finally settle.

Metastability

- When a flip-flop tries to **capture a signal that's changing**, the **timing** between the signal change and the clock edge matters a lot.
- If the signal changes **very close** to the clock edge, the flip-flop has a hard time figuring out whether the signal was a 0 or a 1. The closer the two events happen together, the **longer it takes** for the flip-flop to make a decision.
- If it takes **too long**, the output might not settle to a clear 0 or 1 in time — this is called a **synchronization failure**, caused by **metastability**.



Effects

- If this unstable output is sent to other parts of the logic, such as gates or downstream flip-flops, those circuits might interpret the signal differently — some may see it as a 0, while others may see it as a 1.
- This difference in interpretation can cause the logic to react unpredictably, potentially producing illegal or incorrect outputs.
- As a result, metastability can corrupt data, cause glitches, or activate logic blocks that should not have been triggered.
- These effects can propagate through the rest of the design and lead to unreliable system behavior.

Prevention

- Input to FF must stay stable before or after clock for duration of Setup & Hold time.
- The setup and hold times prevents metastability and keep the circuit working reliably.
- This will ensure flip-flop reads the input correctly.
- If the input changes too close to the clock edge, the flip-flop might get confused and go into a state where the output is not clearly 0 or 1 leading to metastability.

Recommended practices

- CDC paths are often ignored in constraint (XDC) files using `set_false_path` or `set_clock_groups -asynchronous`.
- While this might be okay if the CDC logic is **carefully designed**, it's much better to **define proper CDC timing constraints**.
- Proper constraints helps synthesis and place & route tools keep CDC flip-flops close together.
- Instead of skipping CDC paths, static timing analysis (STA) can check them properly to ensure timing is met.

Recommended techniques

- **Reduce load on CDC flip-flops** – to choose faster flops and shorten connections.
- **Minimize delay between CDC flip-flops** – to avoid tools adding unnecessary buffers.
- Synchronizing signals from the sending clock domain

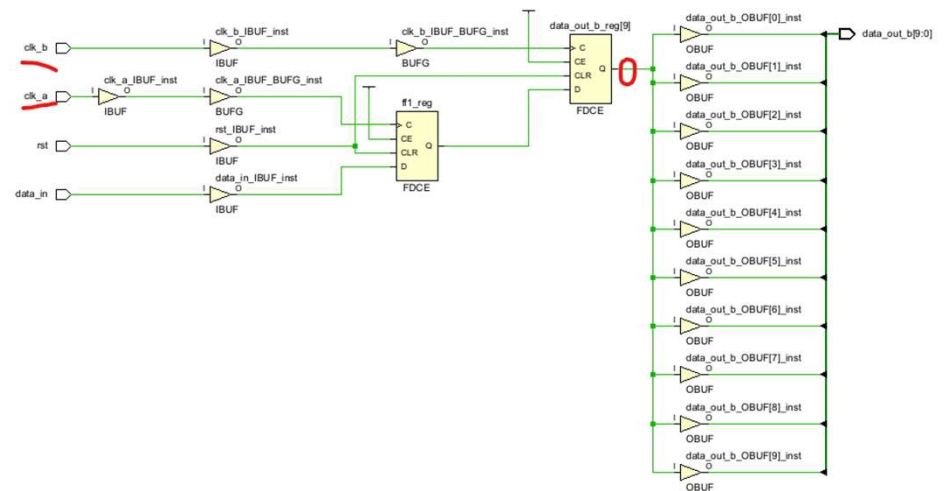
Reduce load on CDC flip-flops

High load on cdc edge

```
reg ff1;

always @(posedge clk_a or posedge rst) begin
    if (rst)
        ff1 <= 1'b0;
    else
        ff1 <= data_in;
end

always @(posedge clk_b or posedge rst) begin
    if (rst)
        data_out_b <= 10'b0;
    else
        data_out_b <= {10{ff1}}; // using ff1 di.
end
```



Approach

- Both flip-flops in the CDC synchronizer — sync_ff1 and sync_ff2 — should ideally have low fanout (low load), but it is especially important for the first flip-flop (sync_ff1) to have minimal load.
- If sync_ff1 has high fanout (i.e., drives many logic gates) then generally stronger slower flipflops will be adopted which **may take longer to stabilize** after metastability.
- This increases the chance that sync_ff2 may sample an unstable signal reducing **Mean Time Between Failures (MTBF)** of your CDC circuit.

```
set_max_fanout 1 [get_cells sync_ff1]
```

Approach

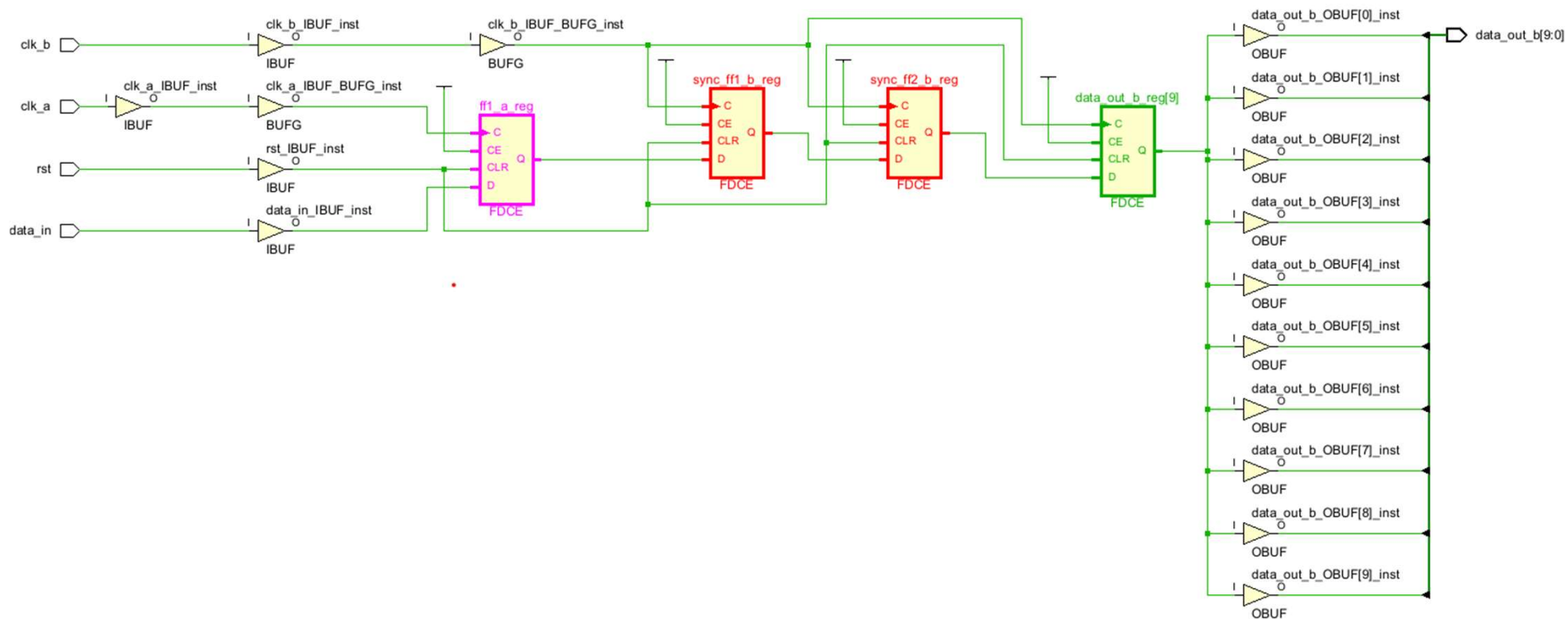
- sync_ff2 samples the (hopefully resolved) signal from sync_ff1.
- Its output is considered safe and stable, so it can drive more logic.
- Still, it's good practice to **avoid driving very heavy logic** directly from it — instead, **buffer or pipeline** it if needed.

```

always @(posedge clk_b or posedge rst) begin
    if (rst) begin
        sync_ff1_b <= 1'b0;
        sync_ff2_b <= 1'b0;
    end else begin
        sync_ff1_b <= ff1_a;
        sync_ff2_b <= sync_ff1_b;
    end
end

// Safe usage of synchronized signal
always @(posedge clk_b or posedge rst) begin
    if (rst)
        data_out_b <= 10'b0;
    else
        data_out_b <= {10{sync_ff2_b}}; // Now safe in clk_b domain
end

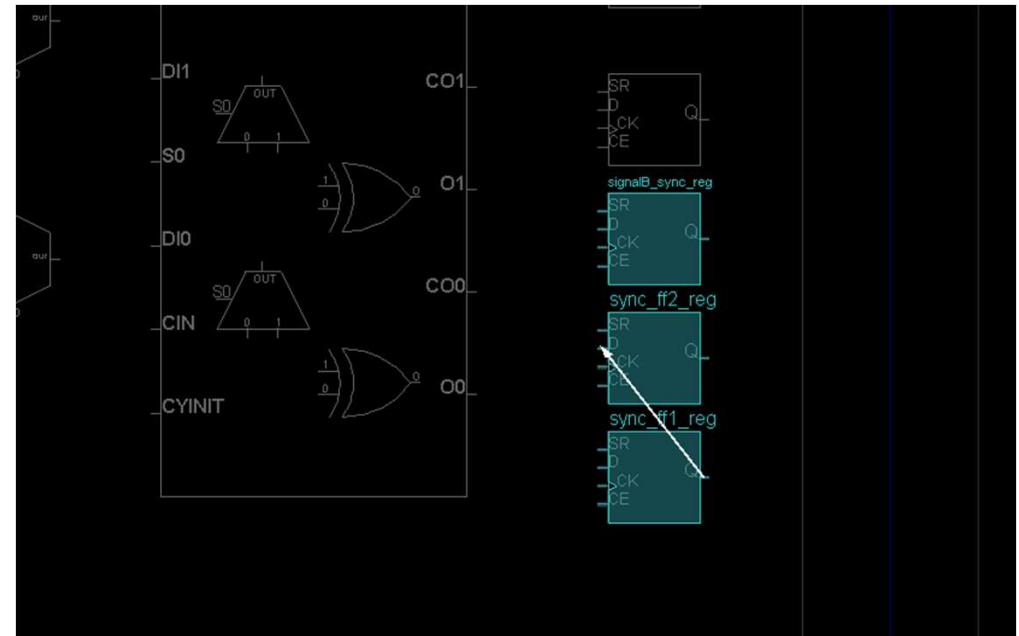
```



Minimize delay between CDC flip-flops

```
// Two-stage synchronizer in clk_b domain
(* ASYNC_REG = "TRUE" *) reg sync_ff1_b;
(* ASYNC_REG = "TRUE" *) reg sync_ff2_b;

always @(posedge clk_b or posedge rst) begin
    if (rst) begin
        sync_ff1_b <= 1'b0;
        sync_ff2_b <= 1'b0;
    end else begin
        sync_ff1_b <= ff1_a;
        sync_ff2_b <= sync_ff1_b;
    end
end
```



Problem with 2-ff sync for multi-bits

- Each bit of the bus may experience **different metastability delays** and **settle at different times**.
- Some bits may be from the **previous value**, and Some bits from the **new value** (already transitioned).
- This causes **bit-skew** — you read a **corrupted, mixed version of the data**.
- **Bus skew** is the condition where **different bits of a multi-bit bus arrive or settle at different times** at the destination, **causing incorrect or inconsistent data to be sampled**.
- Syncing each bit of a bus individually can cause mismatched sampling due to metastability, leading to corrupted data.
- Prefer handshake or FIFO for multi-bit transfers.

Scenario 1: missing few intermediate values won't effect

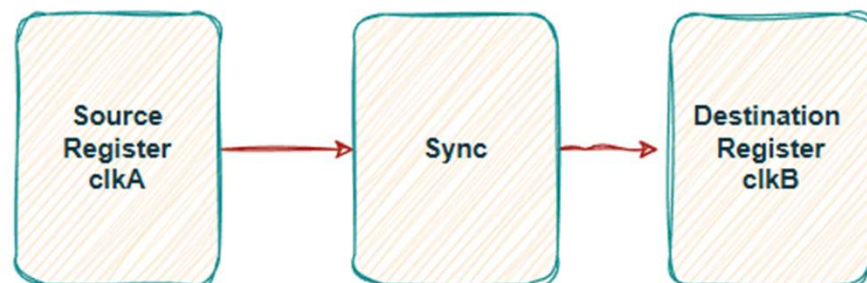
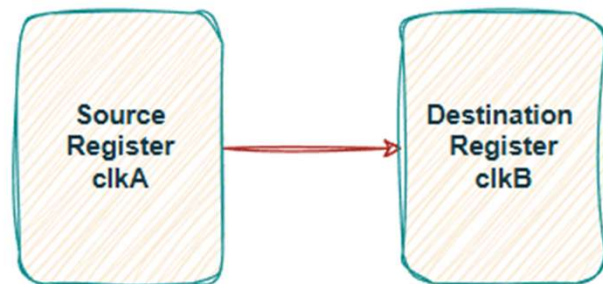
- Temperature sensor updates data in one clock domain (clk_sensor), and a microcontroller reads the data in another clock domain (clk_cpu).
- Variation is small (26.1°C → 26.2°C → 26.3°C → 26.4°C)
- CPU reads 26.1°C → 26.4°C
- CPU skipped 26.2°C and 26.3°C, but the **latest value is correct**, and that's all that matters for display or logging.
- We **don't need to sample every transition but current read value must be accurate.**
- We can adopt simple sync technique like 2-FF Sync

Scenario 2: Must capture all values of signal

- ADC operating at different clock fire interrupt for CPU working at different clock.
- If the CPU misses that signal (e.g., due to bad synchronization), it **misses the received data** entirely.
- Every event or pulse must be captured, If missed, functionality breaks.
- Use a 2-flop synchronizer to safely bring intr into cpu_clk domain.
- Use rising edge detection to create a clean 1-cycle intr_trig pulse.
- Optionally, use an acknowledge signal from cpu to peripheral generating interrupt.

Synchronizer

- A **synchronizer** is a small circuit consisting of two or more **flip-flops** — that helps safely **transfer a signal** from one **clock domain** to another.
- **Primary Objective** of a synchronizer is to **prevent uncertain (metastable) values** — typically voltages **between 0.4 V and 0.8 V** in a 1.2 V logic system — from being passed into the rest of the design.
- **Captures an asynchronous signal** (a signal not aligned with the local clock)
- if it goes metastable (in-between 0 and 1), the first flip-flop may not produce a clean logic level immediately.
- The **second flip-flop waits** one full clock cycle, giving the signal a chance to **resolve to a stable logic level**.
- Only the **clean, resolved value** is then used in the rest of your synchronous logic.



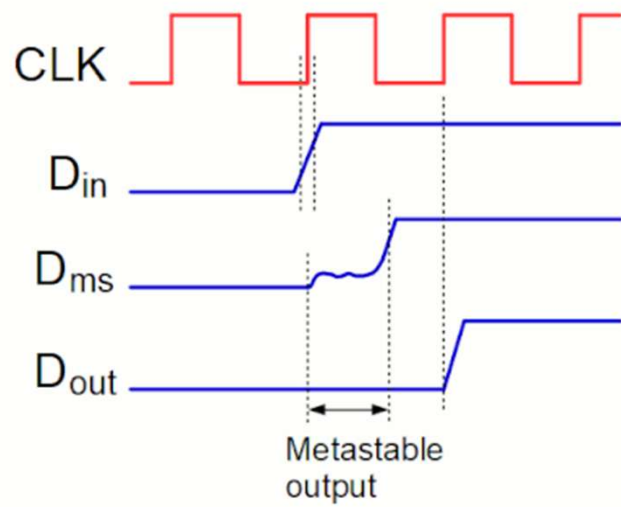
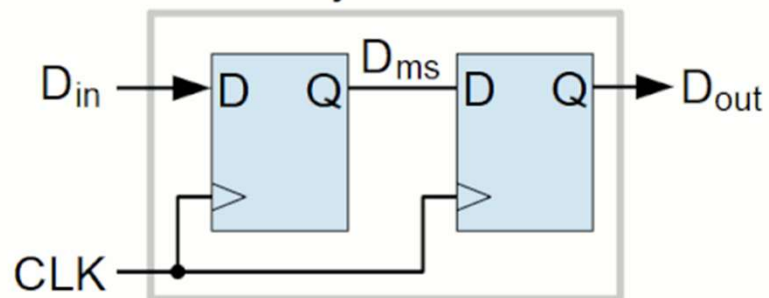
CDC paths

- Single bit CDC : Transfer a flag, pulse, or 1-bit signal from one clock domain to another.
- Use a **2-flop synchronizer** (2FF).
- Multi-bit CDC : Transfer a bus, counter value, or state machine outputs that must arrive **together and consistent**.
- Use FIFO / Handshake mechanism

2-Flop Sync

- The first flip-flop captures the signal coming from another clock/async signal.
- If the signal is unstable, it may take time to settle usually less than period of clock/ few ns. Hence mostly stable withing single clock period.
- The second flip-flop waits one clock cycle, then samples the settled value. This helps ensure the output is stable and safe to use in the new clock domain.
- For most simple synchronization applications, the two flip-flop synchronizer is sufficient to remove all likely metastability

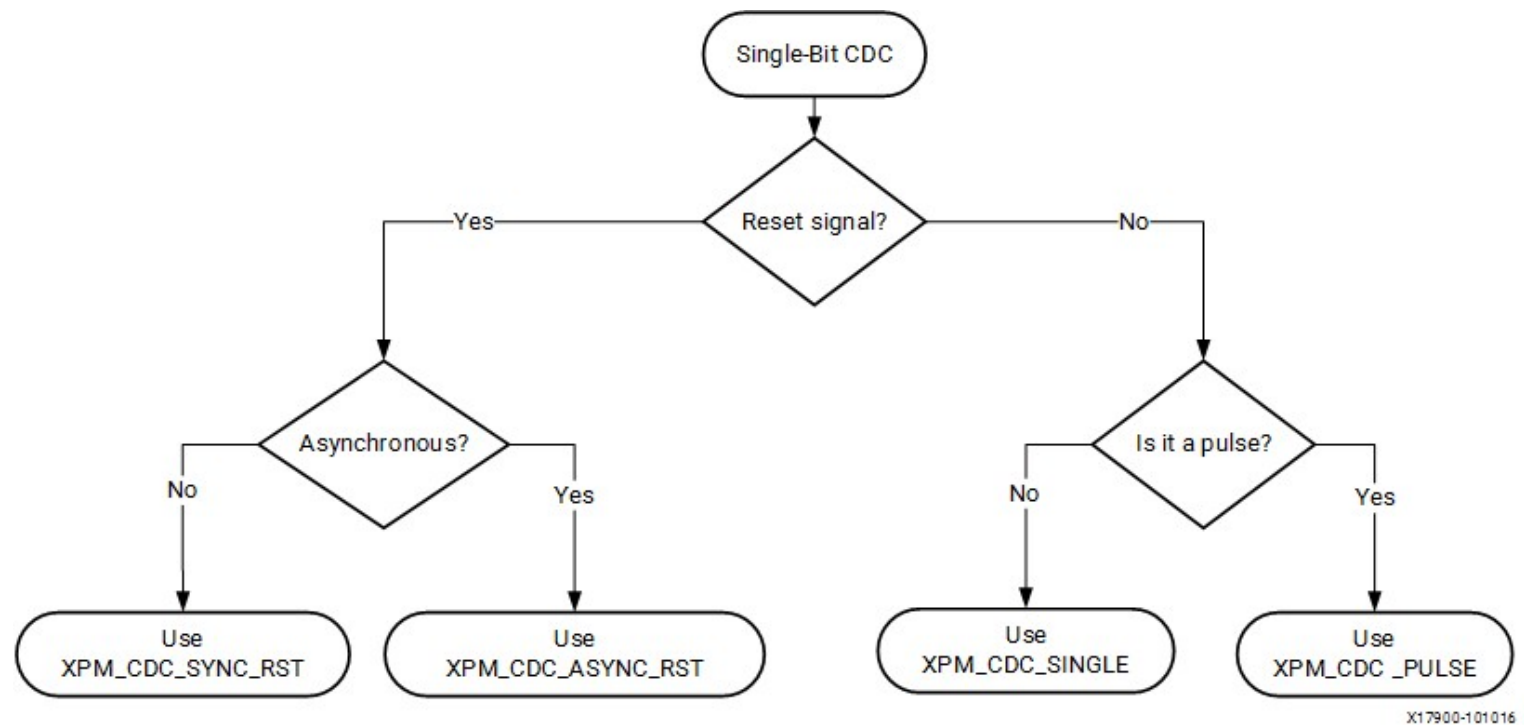
2-FF synchronizer



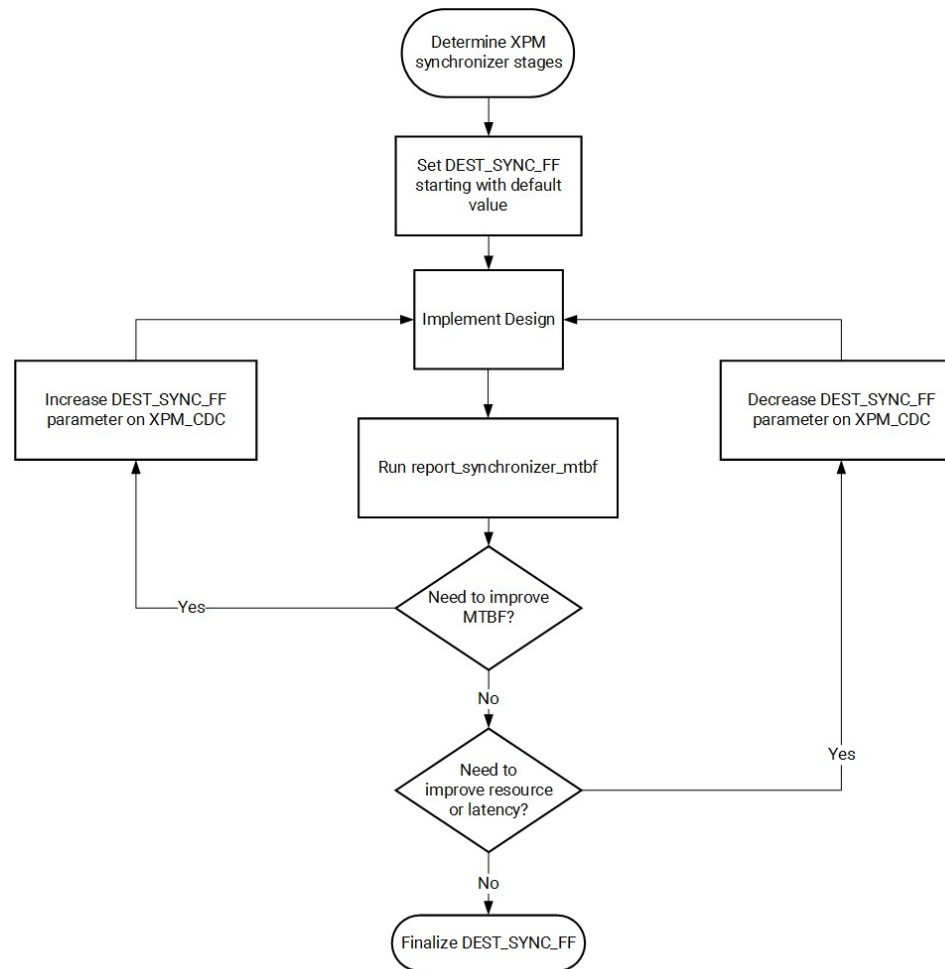
Demo

- CDC flow
- Sensing locked signal in other domain
- Sensing async signal in pther domain

Data Loss & Data Incoherency



You are trying to transfer a **1-bit signal** from one clock domain to another.



X17899-122019

Synchronizer MTBF Optimization Flow for UltraScale Device

MTBF

- **Mean Time Between Failures** is a **statistical measure of reliability** and predicts on average, how long a system will operate before a failure occurs.
- MTBF predicts how much time will pass between two metastability induced failures.
- An MTBF of **1 million hours** means *on average*, one failure will occur every 1 million hours of operation.
- **High MTBF** → system is reliable, very rare failures.
- **Low MTBF** → system may frequently fail due to metastability.

$$MTBF = \frac{e^{S/\tau}}{WF_cF_d}$$

Where : S = Settling time (s)

τ = Settling time constant (s)

F_c = Sampling clock frequency (Hz)

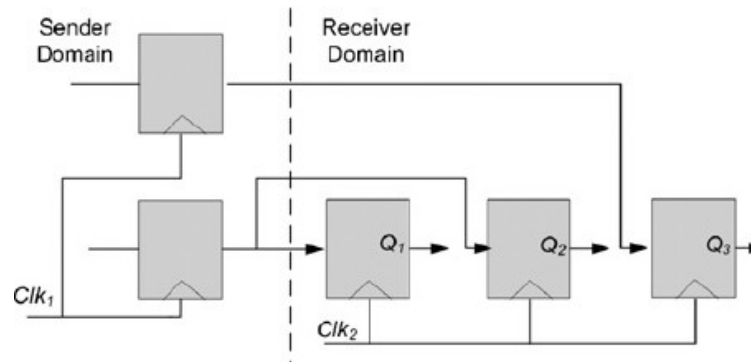
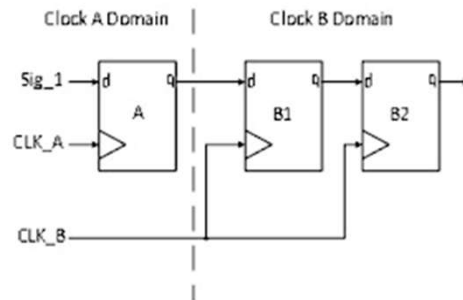
F_d = Change frequency (Hz)

W = Metastability window (s)

- **S : Settling time (in seconds)** – How long the signal is allowed to stabilize before being sampled.
- **τ : Settling time constant (in seconds)** – A technology-dependent parameter describing how fast the circuit settles.
- **F_c : Sampling clock frequency (in Hz)** – Frequency of the clock domain that samples the signal.
- **F_d : Data change frequency (in Hz)** – How often the source (launch) domain changes the data (typically the inverse of the launch clock period multiplied by the toggle rate).
- **W : Metastability window (in seconds)** – The time window during which sampling can lead to metastability.

Add More Synchronizing Flip-Flops (Increase Settling Time S)

- Use a **2-FF or 3-FF synchronizer** in the destination domain.
- Each FF gives more time for metastability to settle.
- Increases $S \rightarrow$ exponentially increases MTBF.



Slow Down the Data Toggle Rate

- Avoid rapidly changing data crossing into another clock domain.
- Use **handshake logic** or **data valid strobes** to limit changes.
- Implement **valid/ack** or **toggle/capture** style CDC.
- Handshakes **avoid metastability**, since the transfer is acknowledged.
- If feasible, use a slower destination clock.
- Choose FFs with **faster recovery time** (lower τ) and **narrower metastability windows** -> reduce load on sync fifo

- <https://github.com/CESNET/ndk-fpga/blob/687d372ee10bdd2bc372ce6cce2734c8c5796823/comp/base/fifo/asfifox/asfifox.vhd#L353>
- http://www.gstitt.ece.ufl.edu/courses/fall11/eel4930_5934/lectures/metastability/276202.pdf

CDC

- Single bit cdc : **use to single-bit signal from one clock domain to another, avoiding metastability** and ensuring correct operation in FPGA and ASIC designs.
- Synchronizing a reset from an async or different clock source.
- Transferring control & status signal across domains.

Approaches

- Use vendor-provided CDC primitives (like Xilinx's XPM_CDC_)
- Manually code synchronization using flip-flops (e.g., 2-stage FF)

```
reg ff1, ff2;  
always @(posedge clk_dst) begin  
    ff1 <= async_signal;  
    ff2 <= ff1;  
end
```

Primitive	Use Case
XPM_CDC_SYNC_RST	Synchronous reset signal
XPM_CDC_ASYNC_RST	Asynchronous reset signal
XPM_CDC_PULSE	One-clock-cycle pulse signal
XPM_CDC_SINGLE	Level signal (e.g., enable flag)

Primitives

- Designed and validated by FPGA vendors (Xilinx, Intel, etc.) with built-in timing constraints and metastability protection.
- Work just perfect with vendors specific fpga families.
- Abstracts the CDC mechanism—you just instantiate and connect ports.
- Easy to use for single-bit, pulse, or even handshake/multi-bit CDC.
- Not portable across different FPGA vendors.

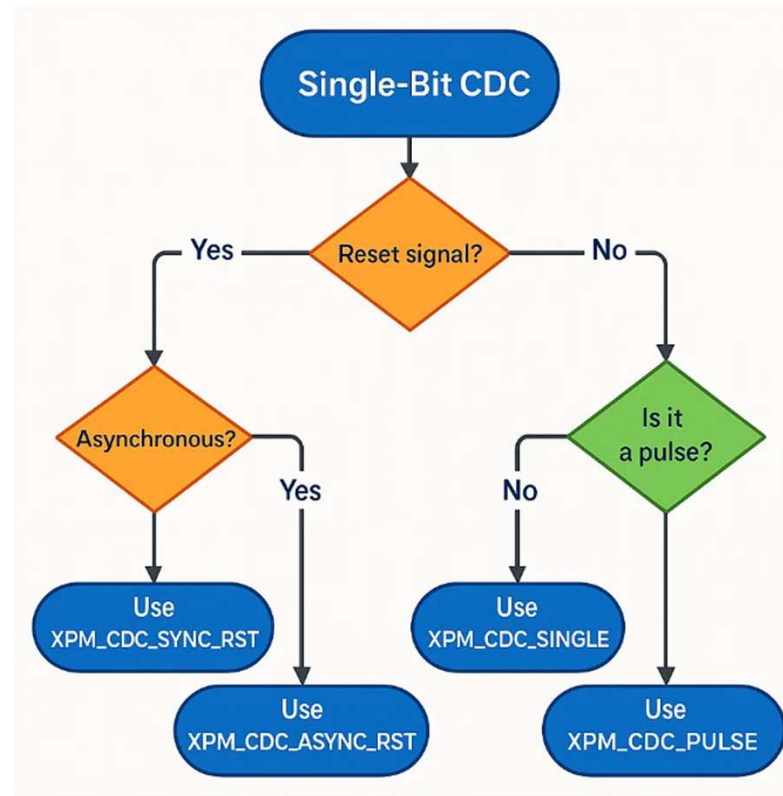
Manual

- Works across all tools and FPGA vendors.
- Full control : You can customize how many stages, add reset, etc.
- Without extra constraints, tools might not know this is a CDC path.
- You must manually add `set_false_path` or `set_max_delay` in SDC/XDC.
- Harder to reuse or scale to pulses or multibit.

Recommendation

- Use **CDC primitives** in production designs (especially for FPGAs like Xilinx).
- Use **manual flip-flop stages** in learning, conceptual designs, or simple/small systems.

Single bit decision tree



XPM_CDC_ASYNC_RST

- Synchronizes an asynchronous reset signal to the destination clock domain.
- Assert asynchronously in relation with input
- De-assertion of the output will always be synchronous to the destination clock domain.



```

always @(posedge clk or posedge ck_rst)
  if (ck_rst) begin
    rst_r <= 1'b1;
    rst   <= 1'b1;
  end else begin
    rst_r <= 1'b0;
    rst   <= rst_r;
  end
end

```

Cycle	ck_rst	rst_r	rst
Reset	1	1	1
1st clk after reset deassertion	0	0	1
2nd clk after reset deassertion	0	0	0

```

xpm_cdc_async_rst #(
    .DEST_SYNC_FF    (4), // DECIMAL; range: 2-10
    .INIT_SYNC_FF    (0), // DECIMAL; 0=disable simulation init values, 1=enable simulation init values
    .RST_ACTIVE_HIGH (1) // DECIMAL; 0=active low reset, 1=active high reset
)
xpm_cdc_async_rst_inst (
    .src_arst  (~ck_rst),
    .dest_arst (rst),
    .dest_clk  (clk)
);

```

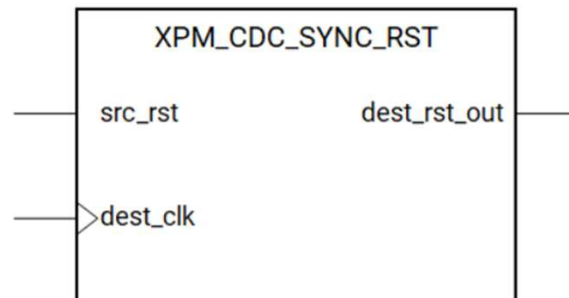
```

xpm_cdc_async_rst #(
    .DEST_SYNC_FF    (4),
    .INIT_SYNC_FF    (0),
    .RST_ACTIVE_HIGH (0) // reset is active-low
)
xpm_cdc_async_rst_inst (
    .src_arst  (ck_rst), // no inversion needed
    .dest_arst (rst),
    .dest_clk  (clk)
);

```

XPM_CDC_SYNC_RST

- synchronizes a reset signal to the destination clock domain.
- Unlike XPM_CDC_ASYNC_RST macro, the generated output will both assert and de-assert synchronously to the destination clock domain.
- Even if a reset is synchronous in its own domain, if you're using that signal in a **different clock domain**, it **must be re-synchronized** to avoid metastability.

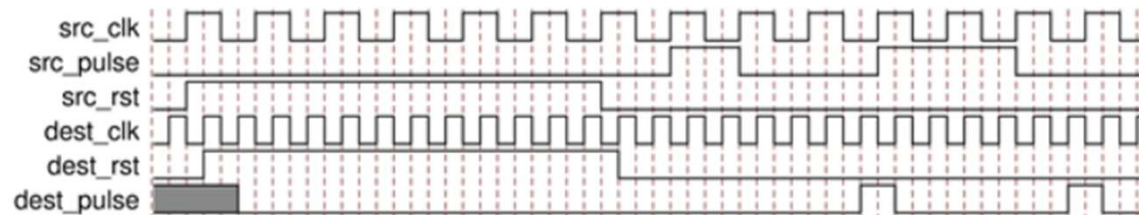


```
xpm_cdc_sync_rst #(
    .DEST_SYNC_FF  (4), // DECIMAL; range: 2-10
    .INIT          (1), // DECIMAL; 0=initialize synchronization registers to 0, 1=initialize synchronization
    .INIT_SYNC_FF  (0), // DECIMAL; 0=disable simulation init values, 1=enable simulation init values
    .SIM_ASSERT_CHK(0) // DECIMAL; 0=disable simulation messages, 1=enable simulation messages
) xpm_cdc_sync_rst_inst (
    .dest_rst(sreset), // 1-bit output: src_rst synchronized to the destination clock domain. This output
    .dest_clk(SCLK ), // 1-bit input: Destination clock.
    .src_rst (RESET ) // 1-bit input: Source reset signal.
);
```

XPM_CDC_PULSE

- synchronizes a pulse in the source clock domain to the destination clock domain.
- A pulse of any size in the source clock domain, if initiated correctly, generates a pulse the size of a single destination clock period.

Figure: Timing for Macro and Transfer Back-to-Back Pulses



The **minimum gap** between pulses must be:
 $2 \times \max(\text{src_clk_period}, \text{dest_clk_period})$

`src_clk` period = 10 ns

`dest_clk` period = 12.5 ns

Max = 12.5 ns

Minimum gap = $2 \times 12.5 \text{ ns} = 25 \text{ ns}$

If using both `src_rst` and `dest_rst_in`, they must be asserted **simultaneously** for:

$((\text{DEST_SYNC_FF} + 2) \times \text{dest_clk_period}) + (2 \times \text{src_clk_period})$

`dest_clk_period` = 12.5 ns

`src_clk_period` = 10 ns

Reset duration = $(3 + 2) \times 12.5 + 2 \times 10$

= $5 \times 12.5 + 20$

= $62.5 + 20 = 82.5 \text{ ns}$

xpm_cdc_single

- Safely transfer of **1-bit control signal** (e.g., a flag, toggle, enable, start, ack) from one clock domain to other.
- Control signal comes from **clock domain A** needs to be consumed reliably in **clock domain B**

xpm_cdc_pulse

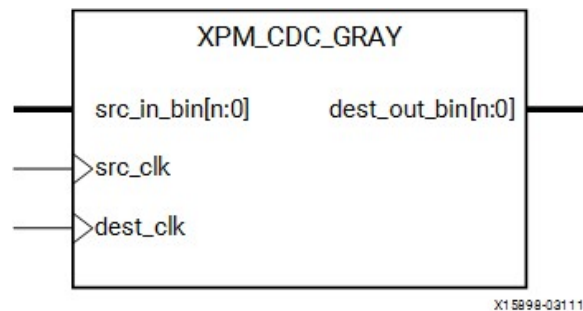
- **Safely transfer a single-cycle pulse** from one clock domain to another.
- xpm_cdc_pulse detects the event in the fast domain (level_in asserted for 1 cycle).
- It toggles an internal flag, which is synchronized to the slow domain (out_clk).
- A one-cycle pulse_out is then generated in out_clk domain.
- This ensures that even a short fast-domain pulse is safely and reliably delivered to the slower domain.

Problem with Single bit sync

- Simple synchronizers are not reliable for transferring multiple signals across clock domains.
- Small differences in signal arrival times can cause bits to be sampled on different clock edges.
- Even with matched trace lengths, rise/fall time differences and process variations can introduce skew.
- Use proper **multi-bit CDC (Clock Domain Crossing) techniques** to avoid sampling errors.

XPM_CDC_GRAY

- Synchronizes a multi-bit **binary input** from one clock domain to another using **Gray code** to avoid metastability.
- Converts binary to Gray code in src_clk domain.
- Synchronizes it using flip-flops in dest_clk domain.
- Converts Gray code back to binary in dest_clk domain.
- Only one bit changes at a time → avoids glitches during clock domain crossing (CDC).



[illegible]

- | | |
|-----------------------------|---|
| - REG_OUTPUT = 0 | Output is combinational . Faster, but may cause glitches. |
| - REG_OUTPUT = 1 | Output is passed through one more flip-flop in <code>dest_clk</code> domain. Ensures stable and glitch-free data. |
| - SIM_LOSSLESS_GRAY_CHK = 1 | Simulation warns if input jumps more than 1 , which violates Gray code safety. |
| - SIM_LOSSLESS_GRAY_CHK = 0 | No warning, even if the Gray coding rule is broken. |

Multi-bit cdc

- Used to transfer multi-bit signals from one asynchronous clock domain to another.
- No guarantee that all bits are sampled from the same clock cycle — bits may reflect different values if source changes too fast / source & destination clocks are close.
- Source data must remain stable for at least 2 cycles of the **destination clock** to avoid corruption.
- Minimal logic — no handshakes, buffers, or control signals.
- Config registers, interrupt vectors, static flags/status updates shared across domains.

Used cases

- Config registers are typically updated by software so they remain constant until explicitly changed and don't change frequently. E.g. setting up baud rate at start of operation.
- Input vectors are often changed individually so we are more of safely transferring single bit rather than word in async domains.
- Status flags like `pll_locked` remain constant for entire duration of time.

```
xpm_cdc_array_single #(
    .DEST_SYNC_FF(4),    // DE
    .INIT_SYNC_FF(0),    // DE
    .SIM_ASSERT_CHK(0),  // DE
    .SRC_INPUT_REG(1),    // DE
    .WIDTH(2)            // DE
)
xpm_cdc_array_single_inst (
    .dest_out(dest_out), // W
                        // C

    .dest_clk(dest_clk), // 1
    .src_clk(src_clk),   // 1
    .src_in(src_in)      // W
                        // C
                        // I
                        // 1
```

Parameters

DEST_SYNC_FF	Number of flip-flop stages in the destination clock domain	2 or 3	More stages → better metastability protection
INIT_SYNC_FF	Whether flip-flops get an initial value in sim	0 (off)	Use 1 if you want initial values to be set in behavioral simulation
SIM_ASSERT_CHK	Enable warnings in simulation about incorrect usage	1 (on)	Helpful for catching errors
SRC_INPUT_REG	Whether to register input using <code>src_clk</code>	0 or 1	Set 1 if input is not already registered
WIDTH	Number of bits in <code>src_in</code> signal	1-1024	Each bit is synchronized independently

Port map

```
xpm_cdc_array_single #(
    .DEST_SYNC_FF(4),    // DE
    .INIT_SYNC_FF(0),    // DE
    .SIM_ASSERT_CHK(0),  // DE
    .SRC_INPUT_REG(1),   // DE
    .WIDTH(2)            // DE
)
xpm_cdc_array_single_inst (
    .dest_out(dest_out), // W
                        // C

    .dest_clk(dest_clk), // 1
    .src_clk(src_clk),   // 1
    .src_in(src_in)      // W
                        // C
                        // I
                        // 1
```

dest_clk

Input

Clock in the **destination** domain — the domain you're moving to

src_clk

Input

Clock in the **source** domain — needed only if `SRC_INPUT_REG = 1`

src_in

Input

Multi-bit signal you want to transfer — each bit is treated independently

dest_out

Output

The synchronized version of `src_in` — safe to use in `dest_clk` domain

Used Cases

- When transferring **flags, status bits, interrupt vectors, or slow-changing config registers**.
- Each bit can be treated independently — you **don't need full-word coherence**.

Data incoherency

- If multiple signals are sent together and each is synchronized separately using flip-flops, they might not all arrive at the same time in the destination domain.
- some might get captured earlier than others due to clock edge alignment and metastability effects.
- This causes the destination domain to see a mix of old and new values — a condition known as **loss of data coherence**.
- If these signals control a function together, such an invalid combination can lead to errors.

XPM_CDC_GRAY

- Data incoherency happened because **not all bits of the bus changed at the same time** in the destination clock domain.
- Some bits changed earlier, while others changed later, leading to an **invalid intermediate value**.
- if we design the system such that **only one bit changes during each state transition**, then even if that bit gets delayed or captured differently, the destination bus would still hold either the old value or the new one — **never an invalid combination**.
- So, when Gray-coded data crosses clock domains, you avoid invalid intermediate values — **solving the data coherence problem**.

Used cases

- xpm_cdc_gray is used to safely transfer Gray-coded multi-bit values across asynchronous clock domains.
- In asynchronous FIFO designs, the **write pointer** needs to be safely passed to the **read domain**, and vice versa.
- Transferring a frame count, packet ID, or address from a slower clock domain to a faster one without risk of bit mismatch.

xpm_cdc_gray # (	DEST_SYNC_FF	Number of sync flip-flops	Determines how many flip-flops are used in the destination clock domain to safely synchronize the Gray-coded input. Typically set to 2 or more to protect against metastability.
.DEST_SYNC_FF(4), //			
.INIT_SYNC_FF(0), //	INIT_SYNC_FF	Initial value in simulation	When set to 1 , it initializes the sync flip-flops to a known value during behavioral simulation. This has no effect on actual hardware .
.REG_OUTPUT(0), //			
.SIM_ASSERT_CHK(0), //			
.SIM_LOSSLESS_GRAY_CHK(0), //			
.WIDTH(2) //			
)			
xpm_cdc_gray_inst (	REG_OUTPUT	Output register enable	When set to 1 , the synchronized binary output (<code>dest_out_bin</code>) is registered (i.e., delayed by one more clock cycle for stability). When 0 , it is combinational.
.dest_out_bin(dest_out_bin),			
	SIM_ASSERT_CHK	Simulation checker	Enables or disables runtime checks during simulation to catch incorrect usage of the CDC module (like clock domain errors).
.dest_clk(dest_clk),			
.src_clk(src_clk),			
.src_in_bin(src_in_bin)	SIM_LOSSLESS_GRAY_CHK	Simulation checker for Gray transitions	Checks during simulation whether the input (<code>src_in_bin</code>) is changing in Gray-code style i.e., only one bit changes at a time , ensuring safe CDC.
	WIDTH	Number of bits	Sets the width of the binary signal being transferred. Allowed range is 2 to 32 bits. This signal is automatically converted to Gray code during transfer.
);			

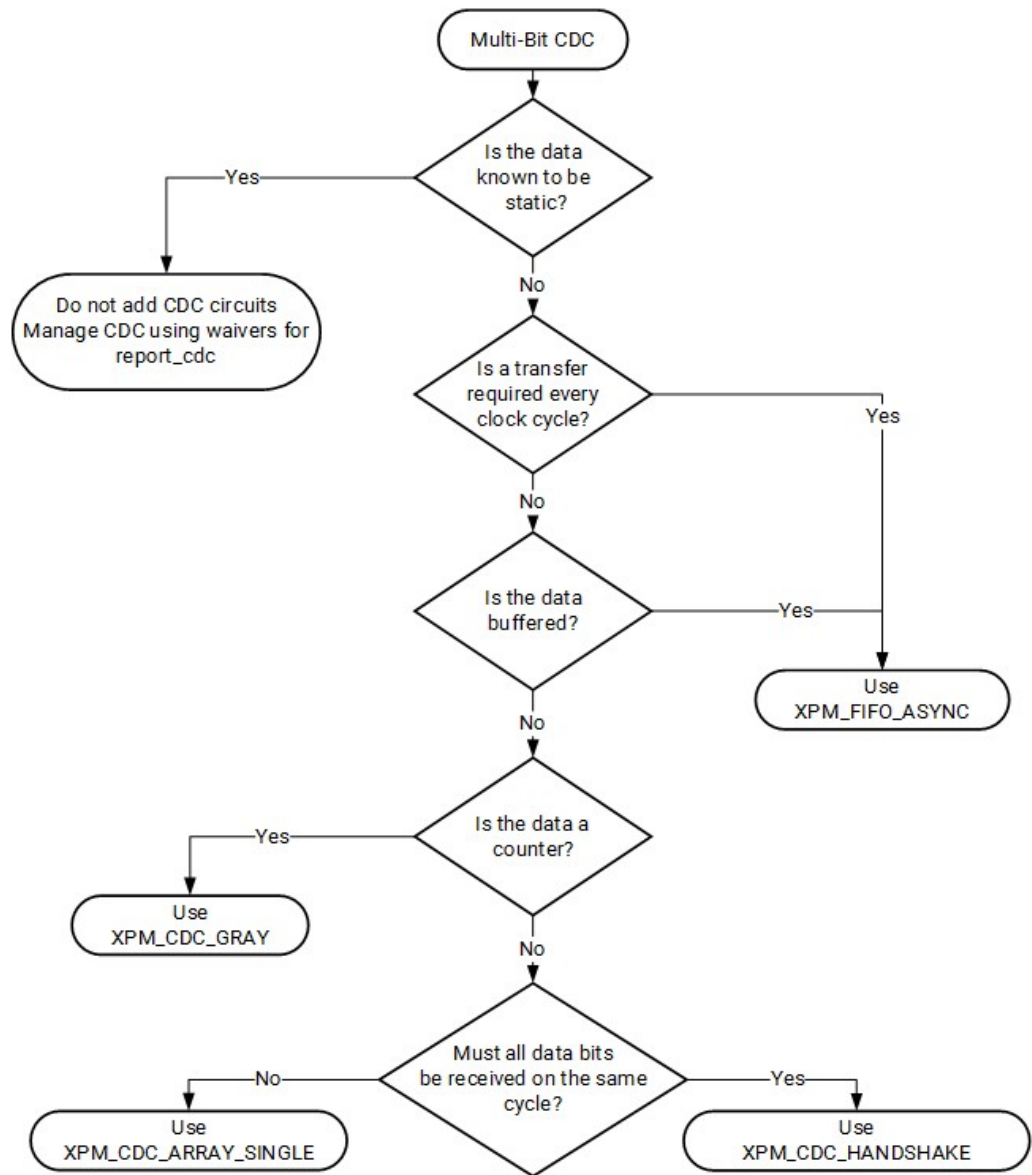
```
xpm_cdc_gray #(
    .DEST_SYNC_FF(4),           //
    .INIT_SYNC_FF(0),          //
    .REG_OUTPUT(0),             //
    .SIM_ASSERT_CHK(0),        //
    .SIM_LOSSLESS_GRAY_CHK(0), //
    .WIDTH(2)                  //
)
xpm_cdc_gray_inst (
    .dest_out_bin(dest_out_bin),

    .dest_clk(dest_clk),
    .src_clk(src_clk),
    .src_in_bin(src_in_bin)
);
```

dest_clk	Input clock (destination)	Clock of the domain where the data is being received.
src_clk	Input clock (source)	Clock of the domain where the binary input is generated.
src_in_bin	Input binary data	Multi-bit binary value in the source domain that you want to transfer safely. Internally, it is converted to Gray code , synchronized, then converted back to binary.
dest_out_bin	Output binary data	The synchronized version of <code>src_in_bin</code> , converted back to binary in the destination domain. May be registered or combinational depending on <code>REG_OUTPUT</code> .

Used cases

- `xpm_cdc_gray` is used when you need to transfer a multi-bit counter or pointer from one clock domain to another safely.
- It converts the binary input to Gray code, synchronizes it bit-by-bit using flip-flops, then converts it back to binary in the destination domain.



Primitive	When to choose
<code>xpm_cdc_gray</code>	Use for transferring counters or pointers safely between clock domains.
<code>xpm_cdc_array_single</code>	Use for transferring small multi-bit control or status signals without handshake.
<code>xpm_cdc_handshake</code>	Use for transferring single events or control words with guaranteed delivery using handshake.
<code>xpm_fifo_async</code>	Use for transferring streams or bursts of data with buffering across clock domains.

- `xpm_cdc_array_single` → Use when you want to transfer control bits without waiting for confirmation.
- `xpm_cdc_handshake` → Use when you want confirmation that your event or command was safely received.

Handshake vs fifo

✓ Use `XPM_CDC_HANDSHAKE` when...

You need to send **single bits or small control signals**.

The source and destination both need to agree on each transfer (tight handshake).

Transfer rate is **low / occasional**.

No need to buffer multiple values.

Example: trigger, start pulse, status flag.

✓ Use `xpm_fifo_async` when...

You need to send **data streams or multiple words**.

The producer and consumer work independently (loose coupling).

Transfer rate is **high / bursty / continuous**.

Need to buffer multiple values to handle rate mismatch.

Example: packets, samples, payload data.

Use `XPM_CDC_HANDSHAKE` → for *single events, flags, or low-rate control signals* crossing clock domains.

Use `xpm_fifo_async` → for *bulk data, streams, or high-speed data* crossing clock domains.

XPM_FIFO_ASYNC

- This macro is used to create an **asynchronous FIFO** — a memory buffer that passes data between **two different clock domains**.
- After a reset, wait until `wr_rst_busy` and `rd_rst_busy` go low before doing anything.

Writing Data

- Happens on each `wr_clk` edge if:
 - FIFO is **not full**.
 - `wr_en` is high.

Reading Data

- Happens on each `rd_clk` edge if:
 - FIFO is **not empty**.
 - `rd_en` is high.

Don't toggle `wr_en` or `rd_en` when:

- `rst` is active.
- `wr_rst_busy` or `rd_rst_busy` is high.

Operation

- After a user issues a reset, the user should wait until the busy signals go low before issuing another reset.
- All synchronous signals are sensitive to the rising edge of wr_clk/rd_clk.
- A write operation is performed when the FIFO is not full and wr_en is asserted on each wr_clk cycle.
- A read operation is performed when the FIFO is not empty and rd_en is asserted on each rd_clk cycle.

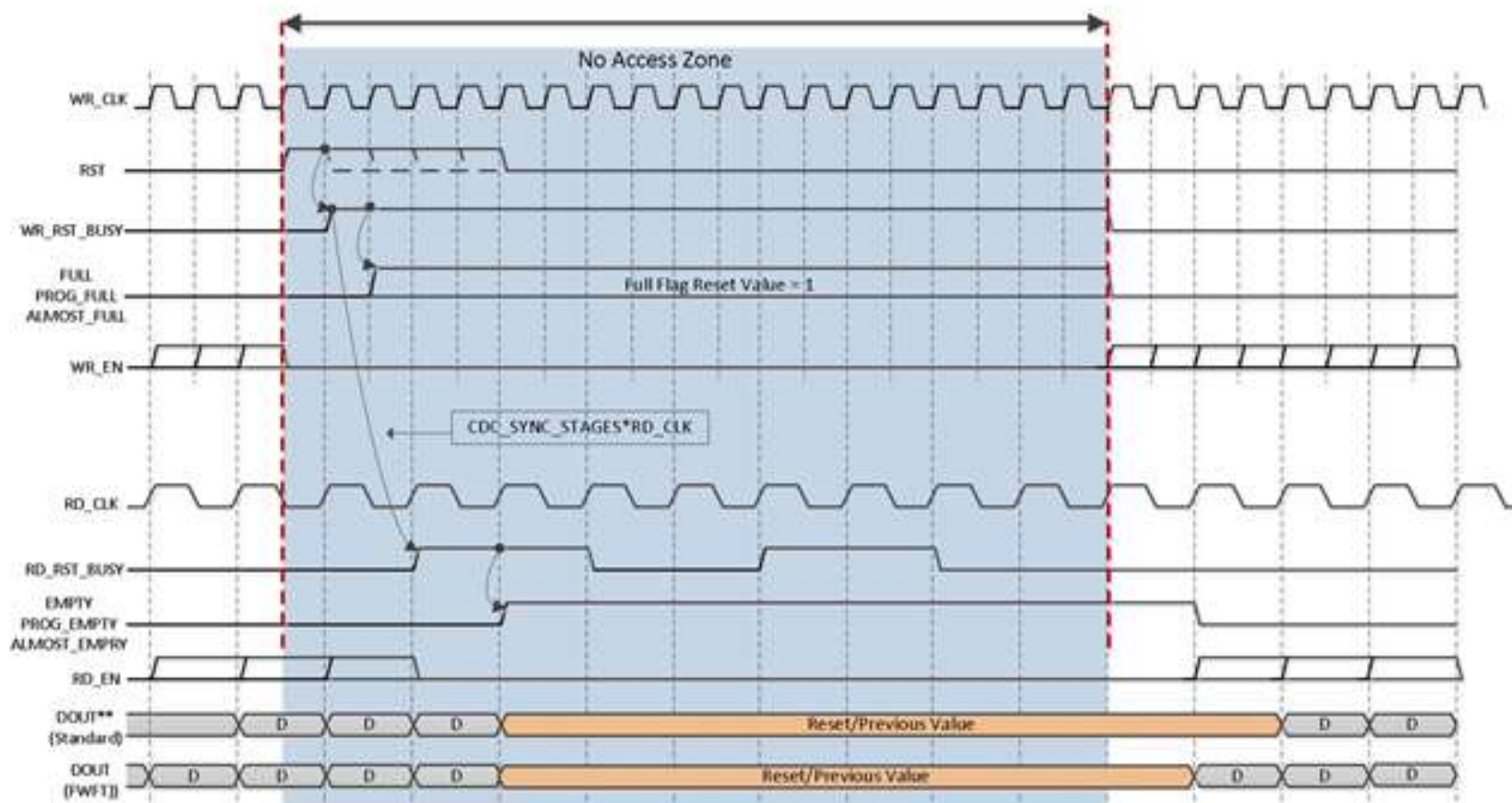
```

xpm_fifo_async #(
    .FIFO_MEMORY_TYPE    ("auto"),    // Memory type: "auto", "block", "distributed", "ultra"
    .FIFO_READ_LATENCY   (0),         // 0 or appropriate value based on READ_MODE
    .FIFO_WRITE_DEPTH    (4096),      // FIFO depth
    .READ_DATA_WIDTH     (5),         // Width of dout
    .READ_MODE           ("fwft"),    // "std" or "fwft"
    .USE_ADV_FEATURES     ("1000"),    // Basic feature: no prog_full/empty, no counts
    .WRITE_DATA_WIDTH     (5)         // Width of din
) xpm_fifo_async_inst (
    // Mandatory write domain
    .wr_clk   (wr_clk),    // input: write clock
    .wr_en    (wr_en),     // input: write enable
    .din       (din),      // input [WRITE_DATA_WIDTH-1:0]: data input

    // Mandatory read domain
    .rd_clk   (rd_clk),    // input: read clock
    .rd_en    (rd_en),     // input: read enable
    .dout      (dout),     // output [READ_DATA_WIDTH-1:0]: data output
    .data_valid (data_valid), // output: valid signal in fwft mode

    // Mandatory reset
    .rst       (rst)       // input: active-high reset
);

```



**FIFO_READ_LATENCY = 1

- The number of clock cycles required for XPM FIFO to react to dout, full, and empty changes depends on the CLOCK_DOMAIN, READ_MODE, and FIFO_READ_LATENCY settings.

FIFO_READ_LATENCY

- 0: No extra latency (when using fwft mode, because data appears as soon as it's available)
- 1 or higher: Adds extra cycles before dout is valid

```
.FIFO_READ_LATENCY(0) // Immediate data available (fwft mode)
.FIFO_READ_LATENCY(1) // One extra clock cycle latency after rd_en
```

Figure: Standard Read Operation.

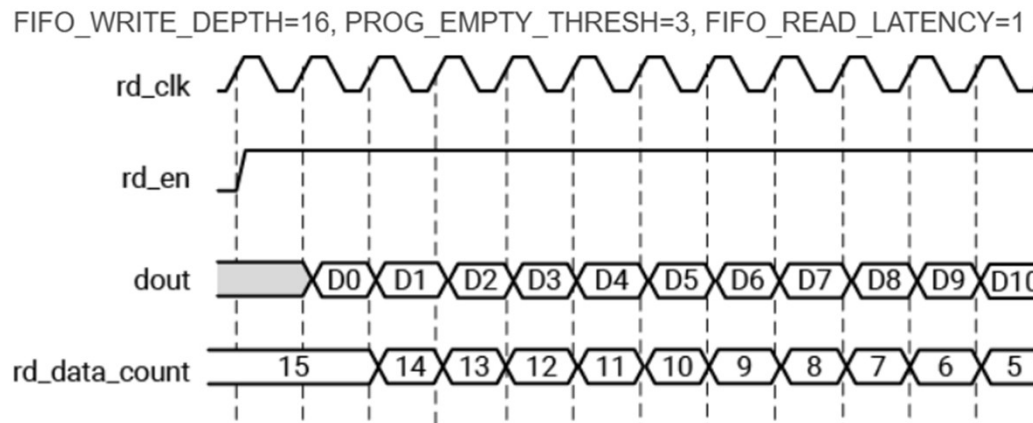


Figure: Standard Read Operation.

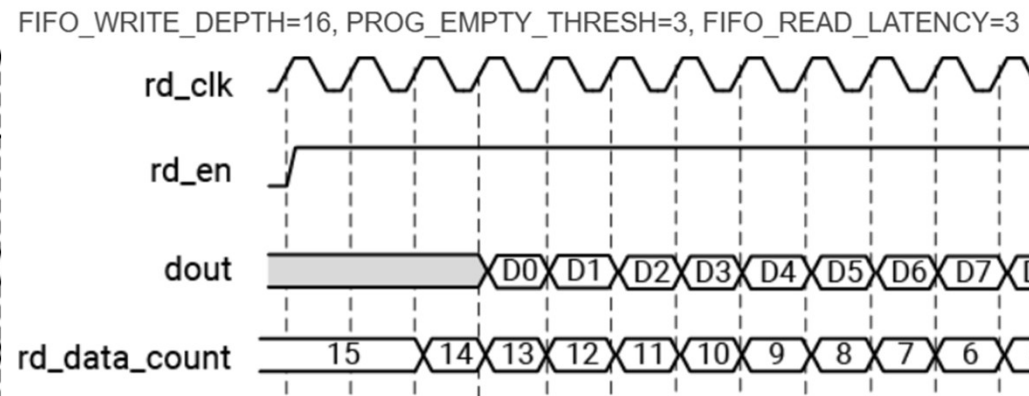
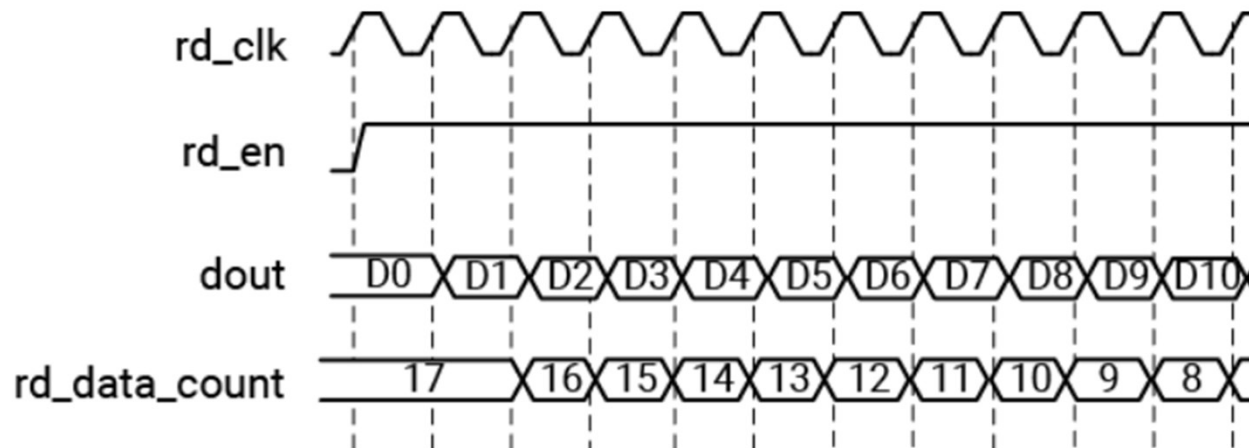


Figure: Read Operation.

READ_MODE=FWFT, FIFO_WRITE_DEPTH=16, PROG_EMPTY_THRESH=5



```

xpm_fifo_async #(
    .FIFO_MEMORY_TYPE    ("auto"),    // 1
    .FIFO_READ_LATENCY   (0),          // 1
    .FIFO_WRITE_DEPTH    (4096),      // 1
    .READ_DATA_WIDTH     (5),         // 1
    .READ_MODE           ("fwft"),    // 1
    .USE_ADV_FEATURES    ("1000"),    // 1
    .WRITE_DATA_WIDTH    (5)          // 1
) xpm_fifo_async_inst (
    // Mandatory write domain
    .wr_clk   (wr_clk),    // input: w
    .wr_en    (wr_en),    // input: w
    .din      (din),       // input [W

    // Mandatory read domain
    .rd_clk   (rd_clk),    // input: r
    .rd_en    (rd_en),    // input: r
    .dout     (dout),      // output [
    .data_valid (data_valid), // output

    // Mandatory reset
    .rst      (rst)        // input: a
);

```

- **"std" — Standard mode**
 - Data appears on `dout` only *after* you assert `rd_en`
 - You must request a read → data comes in next cycle (or after latency)
- **"fwft" — First-Word Fall-Through mode**
 - The first word automatically appears on `dout` without asserting `rd_en`
 - `rd_en` advances the FIFO to the next word
 - `data_valid` tells you when `dout` has valid data

```
xpm_fifo_async #(
    .FIFO_MEMORY_TYPE    ("auto"),    // 1
    .FIFO_READ_LATENCY   (0),         // 1
    .FIFO_WRITE_DEPTH    (4096),      // 1
    .READ_DATA_WIDTH     (5),         // 1
    .READ_MODE           ("fwft"),    // 1
    .USE_ADV_FEATURES    ("1000"),    // 1
    .WRITE_DATA_WIDTH    (5)          // 1
) xpm_fifo_async_inst (
    // Mandatory write domain
    .wr_clk   (wr_clk),    // input: write clock
    .wr_en    (wr_en),     // input: write enable
    .din      (din),       // input [width-1:0]

    // Mandatory read domain
    .rd_clk   (rd_clk),    // input: read clock
    .rd_en    (rd_en),     // input: read enable
    .dout     (dout),       // output [width-1:0]
    .data_valid (data_valid), // output: data valid

    // Mandatory reset
    .rst      (rst)        // input: asynchronous reset
);
```

USE_ADV_FEATURES = "ABCD" (each character is 0 or 1)

Each bit controls:

Bit	Feature
A	Programmable Empty (<code>prog_empty</code>)
B	Programmable Full (<code>prog_full</code>)
C	Overflow / Underflow detection
D	Data count (read/write count outputs)

<code>prog_empty</code>	Early warning — lets your logic act before FIFO empties	When data count $\leq$ threshold
<code>underflow</code>	Error signal — indicates an invalid read	When <code>rd_en</code> asserted while empty
<code>prog_full</code>	Early warning before FIFO is full	When data count $\geq$ threshold
<code>overflow</code>	Error signal — indicates invalid write	When <code>wr_en</code> asserted while full

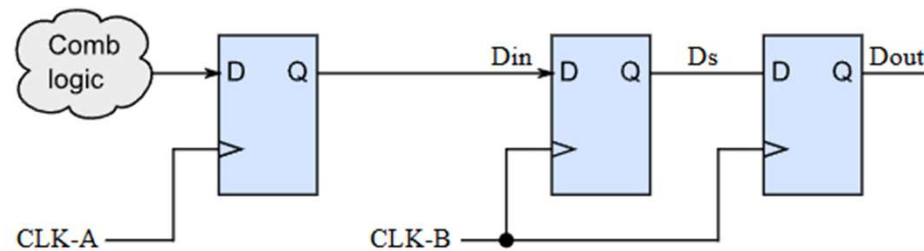
XPM_CDC_SINGLE

- Safely transfer **single-bit signals** across **asynchronous clock domains**.
- Implements a **synchronizer chain** — a sequence of flip-flops — to reduce the risk of metastability when a signal crosses from one clock domain to another.
- Commonly used to transfer **single-bit control signals** (e.g., resets, enables, handshakes) between unrelated or asynchronous clock domains.
- For proper operation, the input data must be sampled two or more times by the destination clock.
- An optional input register may be used to register the input in the source clock domain prior to it being synchronized.
- You can also enable a simulation feature to generate messages to report any potential misuse of the macro.

```

xpm_cdc_single #(
    .DEST_SYNC_FF(4),    // DECIMAL; range: 2-10
    .INIT_SYNC_FF(0),    // DECIMAL; 0=disable simulation init values, 1=enable simulation init values
    .SIM_ASSERT_CHK(0),  // DECIMAL; 0=disable simulation messages, 1=enable simulation messages
    .SRC_INPUT_REG(1)    // DECIMAL; 0=do not register input, 1=register input
)
xpm_cdc_single_inst (
    .dest_out(dest_out), // 1-bit output: src_in synchronized to the destination clock domain. This output
                          // is registered.
    .dest_clk(dest_clk), // 1-bit input: Clock signal for the destination clock domain.
    .src_clk(src_clk),   // 1-bit input: optional; required when SRC_INPUT_REG = 1
    .src_in(src_in)      // 1-bit input: Input signal to be synchronized to dest_clk domain.
);

```



Parameter	Value	Meaning
DEST_SYNC_FF	4	The synchronizer chain uses 4 flip-flops in the destination clock domain. This gives very strong metastability protection (typically 2 is sufficient; 3 or 4 is used in critical cases).
INIT_SYNC_FF	0	The synchronizer flip-flops do not have an initial value in simulation (for RTL simulation cleanliness; hardware init doesn't rely on this).
SIM_ASSERT_CHK	0	No simulation-time assertions or warnings about CDC issues will be generated. (Useful if you want a cleaner sim without extra messages.)
SRC_INPUT_REG	1	A register stage is added at the input (<code>src_in</code>) in the source clock domain (<code>src_clk</code>). This helps reduce hold-time violations on <code>src_in</code> and cleans up the signal before crossing the domain.

Port	Dir	Function
dest_out	output	The synchronized single-bit signal , fully transferred into <code>dest_clk</code> domain. Registered output of the synchronizer chain.
dest_clk	input	Clock of the destination domain.
src_clk	input	Clock of the source domain. Required because <code>SRC_INPUT_REG=1</code> (it needs a clock to register <code>src_in</code>).
src_in	input	The single-bit signal in the source domain you want to transfer safely.

XPM_CDC_PULSE

- This macro synchronizes a pulse in the source clock domain to the destination clock domain.
- For proper operation, the input data must be sampled two or more times by the destination clock.
- You can define the number of register stages used in the synchronizers.
- An optional source and destination reset may be used to reset the pulse transfer logic.
- You can also enable a simulation feature to generate messages which report any potential misuse of the macro.

Template

```
xpm_cdc_pulse #(  
    .DEST_SYNC_FF(4),    // DECIMAL; range: 2-10  
    .INIT_SYNC_FF(0),    // DECIMAL; 0=disable simulation init values, 1=enable simulation init values  
    .REG_OUTPUT(0),       // DECIMAL; 0=disable registered output, 1=enable registered output  
    .RST_USED(1),         // DECIMAL; 0=no reset, 1=implement reset  
    .SIM_ASSERT_CHK(0)    // DECIMAL; 0=disable simulation messages, 1=enable simulation messages  
)
```

```

xpm_cdc_pulse_inst (
    .dest_pulse(dest_pulse), // 1-bit output: Outputs a pulse the size of one dest_clk period when a pulse
transfer is correctly initiated on
                                // src_pulse input. This output is combinatorial unless REG_OUTPUT is set to
1.

    .dest_clk(dest_clk),      // 1-bit input: Destination clock.
    .dest_rst(dest_rst),      // 1-bit input: optional; required when RST_USED = 1
    .src_clk(src_clk),        // 1-bit input: Source clock.
    .src_pulse(src_pulse),    // 1-bit input: Rising edge of this signal initiates a pulse transfer to the
destination clock domain. The minimum gap
                                // between each pulse transfer must be at the minimum 2*(larger(src_clk
period, dest_clk period)). This is measured
                                // between the falling edge of a src_pulse to the rising edge of the next
src_pulse. This minimum gap will guarantee
                                // that each rising edge of src_pulse will generate a pulse the size of one
dest_clk period in the destination clock
                                // domain. When RST_USED = 1, pulse transfers will not be guaranteed while
src_rst and/or dest_rst are asserted.

    .src_rst(src_rst)         // 1-bit input: optional; required when RST_USED = 1
);

```

Minimum Gap between pulses

Formula:

```
java
Minimum Gap = 2 × max(src_clk_period, dest_clk_period)
```

• Example:

- `src_clk_period = 8 ns`
- `dest_clk_period = 5 ns`

Now calculate:

```
java
max(8 ns, 5 ns) = 8 ns
Minimum Gap = 2 × 8 ns = 16 ns
```

• **Meaning:** After a `src_pulse` goes LOW, you must wait at least **16 ns** before asserting the next `src_pulse`.

This ensures the macro has enough time to safely transfer the pulse to the destination clock domain.

- The minimum gap between each pulse transfer must be maintained.
- This is measured between the falling edge of a `src_pulse` to the rising edge of the next `src_pulse`.
- This minimum gap will guarantee that each rising edge of `src_pulse` will generate a pulse the size of one `dest_clk` period in the destination clock domain.

Minimum reset duration

Formula:

```
pgsql Copy Edit  
  
Reset Time = ((DEST_SYNC_FF + 2) × dest_clk_period) + (2 × src_clk_period)
```

Let's say:

- `DEST_SYNC_FF` = 2 (typical value)
- `dest_clk_period` = 5 ns
- `src_clk_period` = 8 ns

Now plug into the formula:

```
pgsql Copy Edit  
  
Reset Time = ((2 + 2) × 5 ns) + (2 × 8 ns)  
            = (4 × 5 ns) + 16 ns  
            = 20 ns + 16 ns  
            = 36 ns
```

Meaning:

When using `src_rst` and `dest_rst`, keep both asserted for at least 36 ns.

During this time:

- `src_pulse` must not toggle.
- The output pulse is not valid — ignore it.

